

Дополнительные материалы

Стиль кода

Содержание

Названия файлов	1
Названия объектов	1
Использование пробелов	2
Фигурные скобки	3
Длина строки	3
Отступы	3
Присваивание	4
Правила комментирования	4

Материал представляет собой сокращённый перевод статей про стиль кода в R:

- [Style guide](#);
- [Google's R Style Guide](#).

У R, как и у других языков программирования, существуют свои стандарты оформления кода. Разберём некоторые основные правила, разработанные Google.

Названия файлов

Названия файлов должны быть содержательными и заканчиваться расширением `.R`. Если файлы с кодом должны открываться или запускаться последовательно, название файла лучше начинать с числа:

```
0-download.R
1-parse.R
2-explore.R
```

Названия объектов

Названия переменных и функций должны записываться маленькими буквами. Для разделения слов в названии объекта нужно использовать символ `_` (символ `.` технически возможен, но нежелателен, так как этот символ используется при написании методов в системе [S3](#)).

```
#
day_one
day_1

#
first_day_of_the_month
DayOne
dayone
djm1
```

Если есть возможность, избегайте использования названий уже существующих функций и переменных.

```
#
T <- FALSE
c <- 10
mean <- function(x) sum(x)
```

Использование пробелов

Добавляйте пробелы вокруг операторов (=, +, -, <-, и т. д.). То же правило работает и при использовании = при вызове функций. Всегда ставьте пробелы после запятых и никогда — перед ними.

```
#
average <- mean(feet / 12 + inches, na.rm = TRUE)
```

```
#
average<-mean(feet/12+inches,na.rm=TRUE)
```

У этого правила существует маленькое исключение: операторы :, :: и ::: не требуют вокруг себя пробелов.

```
#
x <- 1:10
base::get
```

```
#
x <- 1 : 10
base :: get
```

Добавляйте пробел перед левой скобкой в случае, если вы не вызываете какую-либо функцию.

```
#
if (debug) do(x)
plot(x, y)
```

```
#
if(debug)do(x)
plot (x, y)
```

Лишние отступы (более одного пробела в строке) допустимы, если это улучшает выравнивание символов = или оператора присваивания <-.

```
list(
  total = a + b + c,
  mean  = (a + b + c) / n
)
```

Не добавляйте пробелы вокруг кода в скобках или внутри квадратных скобок, если там нет запятой.

```
#
if (debug) do(x)
diamonds[5, ]
```

```
#
if ( debug ) do(x) # debug
x[1,] #
x[1 ,] # ,
```

Фигурные скобки

Открывающая фигурная скобка не должна стоять на отдельной строке, и после неё всегда должен быть переход на новую строку. Закрывающая фигурная скобка должна стоять на отдельной строке в случаях, если после неё не идёт `else`. Всегда добавляйте отступы перед кодом внутри фигурных скобок.

```
#  
  
if (y < 0 && debug) {  
  message("Y is negative")  
}
```

```
if (y == 0) {  
  log(x)  
} else {  
  y ^ x  
}
```

```
#  
  
if (y < 0 && debug)  
message("Y is negative")
```

```
if (y == 0) {  
  log(x)  
}  
else {  
  y ^ x  
}
```

Допустимо писать короткие выражения в одну строку:

```
if (y < 0 && debug) message("Y is negative")
```

Длина строки

Старайтесь, чтобы в одной строке кода было не более 80 символов.

Отступы

Делая отступы в коде (*indentation*), используйте два пробела. Не используйте табуляцию или табуляцию и пробелы одновременно.

Единственное исключение — если определение функции растягивается на несколько строк. В этом случае сделайте отступ перед второй строкой, там, где начинается определение функции:

```
long_function_name <- function(a = "a long argument",  
                               b = "another argument",  
                               c = "another long argument") {  
  # As usual code is indented by two spaces.  
}
```

Присваивание

Используйте `<-`, а не `=` для присваивания.

```
#  
x <- 5
```

```
#  
x = 5
```

Правила комментирования

Комментируйте свой код. Каждая строка комментария должна начинаться со специального символа `#` и пробела. Комментарии должны объяснять, почему выполняется та или иная операция, а не что это за операция.

Используйте символы `-` или `=`, чтобы разбить ваш файл на легко читаемые блоки.

```
# Load data -----  
  
# Plot data -----
```